

CASE STUDY DOCUMENT

Real-Time Chat Application

.NET / ASP.NET Core / EF Core / SQL Server

ConnectHub

Connect. Message. Collaborate. In Real Time.

Version 1.0 | 2026 | .NET 8 / ASP.NET Core / EF Core / SQL Server

1. Synopsis

ConnectHub is a full-stack Real-Time Chat Application built on .NET 8 and ASP.NET Core. It enables users to send and receive instant messages through both direct one-to-one conversations and multi-member group chat rooms. Real-time bidirectional communication is powered by ASP.NET Core SignalR, which maintains persistent WebSocket connections through the ChatHub and pushes messages, typing indicators, read receipts, and online/offline presence updates instantly to all connected clients without polling.

The platform is architected as a set of independently deployable ASP.NET Core Web API microservices — User/Auth-Service, Message-Service, ChatRoom-Service, Presence-Service, Notification-Service, and File/Media-Service — all orchestrated through an ASP.NET Core MVC Website-Controller. Entity Framework Core 8 with SQL Server handles all persistent data. A ConcurrentDictionary-backed PresenceService (registered as AddSingleton) tracks live WebSocket connections in-memory.

2. Detailed Requirements

2.1 User & Authentication

- Users register with username, display name, email, and password; or log in via Google/GitHub OAuth (AddAuthentication().AddGoogle() / AddGitHub()).
- JWT Bearer authentication (AddAuthentication().AddJwtBearer()) secures all API and SignalR Hub endpoints.
- Users can update their profile including display name, bio, and avatar image.
- Users can search for other registered users by username or display name.
- Account deactivation (IsActive = false) prevents login without deleting message history.

2.2 Direct Messaging

- Authenticated users can send direct one-to-one messages to any other user via ChatHub.SendDirectMessage().
- Messages are persisted as Message EF Core entities with SenderId, ReceiverId, Content, SentAt, and IsRead.
- Message delivery is real-time via SignalR: sender calls the Hub method; Hub calls Clients.User(receiverId).SendAsync("ReceiveMessage").
- Users can view full direct message history with a specific user, paginated via GetDirectMessages(senderId, receiverId).
- Users can edit a sent message (content updated, IsEdited = true, EditedAt = DateTime.UtcNow).
- Users can delete their own sent messages (IsDeleted = true soft delete; content replaced with deleted message placeholder).
- Users can search through message history by keyword for a specific conversation.

2.3 Group Chat Rooms

- Users can create public or private chat rooms (RoomType: PUBLIC/PRIVATE/DIRECT) with a name, description, and optional avatar.
- Room creator is automatically added as ADMIN role member (RoomMember.Role).
- Users can join public rooms; private rooms require an invite (AddMember by room admin).
- Users can leave a room at any time via ChatHub.LeaveRoom() / LeaveRoom(roomId, userId).
- Room admins can update room details, change member roles (ADMIN/MEMBER), and remove members.
- Users can send messages to a room via ChatHub.SendRoomMessage(); Hub broadcasts to all room members via Clients.Group(roomId.ToString()).
- Users can browse all public rooms and their member counts.

2.4 Real-Time Presence & Status

- When a user connects via SignalR, ChatHub.OnConnectedAsync() calls IPresenceService.UserConnected(userId, connectionId) storing the connection in a ConcurrentDictionary<int, HashSet<string>>.
- When a user disconnects, ChatHub.OnDisconnectedAsync() calls IPresenceService.UserDisconnected() and broadcasts the offline status to all connected users.
- Typing indicator: when a user starts/stops typing, ChatHub.TypingIndicator(recipientId, isTyping) is called and the Hub pushes a real-time typing event to the recipient.
- Read receipts: ChatHub.MarkMessageRead(messageId) updates Message.IsRead = true and Message.ReadAt = DateTime.UtcNow, then pushes a read-receipt event to the sender.
- The online status badge is displayed on user avatars in the contact list, driven by PresenceService.IsUserOnline(userId).

2.5 File & Media Sharing

- Users can upload images, documents, and audio files in chat via POST /api/media/upload (IFormFile).
- Uploaded files are stored in Azure Blob Storage (Azure.Storage.Blobs NuGet) and referenced by BlobUrl in the MediaFile EF Core entity.
- After upload, the BlobUrl is included in a SendMediaMessage Hub call which persists a Message with MediaUrl and MessageType = IMAGE/FILE/AUDIO.
- SAS tokens (Azure Blob SAS) are generated per-file download request via GenerateSasUrl(fileId) for secure access without public container exposure.
- A BackgroundService (IHostedService) runs daily to delete expired MediaFile records (ExpiresAt < DateTime.UtcNow) and their corresponding Blob storage files.

2.6 Notifications

- In-app notifications are dispatched for: new direct message received, mention (@username) in a room, new member joined a room, room admin role granted.
- Each Notification EF Core entity carries RecipientId, Type (MESSAGE/MENTION/ROOM_INVITE/ROLE_CHANGE), RelatedId, and IsRead.
- Unread notification badge count is pushed in real time to the connected client via SignalR when a new Notification is created.

- Email notifications (MailKit NuGet) dispatched for offline users who receive a direct message.
- Admin can broadcast platform-wide notifications to all users.

2.7 Admin Requirements

- Admins can view, suspend (IsActive = false), and permanently delete any user account.
- Admins can view all chat rooms and delete rooms that violate platform policies.
- Admins can view and delete any message across all conversations and rooms.
- Admins can view platform analytics: total users, messages sent, active rooms, daily active connections.
- Admins can view audit logs of all significant user and admin actions.

3. Use Case Diagram

The diagram below captures all actors and use cases within ConnectHub — spanning authentication, direct messaging, group chat rooms, real-time presence and typing indicators, file/media sharing, notifications, and administrative oversight.

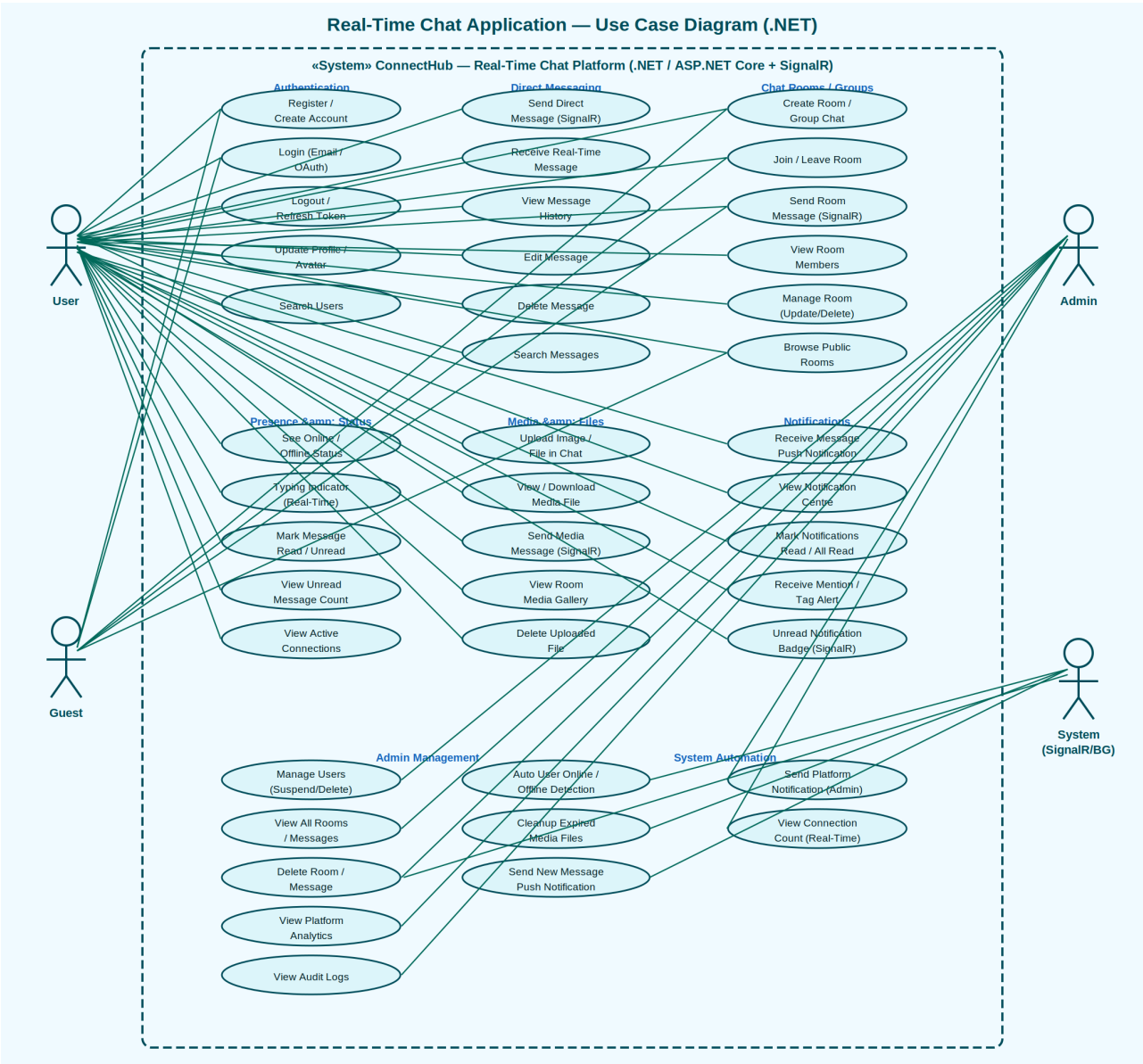


Figure 1: ConnectHub Real-Time Chat Application (.NET + SignalR) — Complete Use Case Diagram

3.1 Actors

Actor	Description
User	Registered user who sends/receives messages, manages rooms, shares files, and tracks presence
Guest	Unauthenticated visitor who can access the registration and login pages only
Admin	Platform administrator managing users, rooms, messages, analytics, and broadcast notifications
System (SignalR/BG)	SignalR Hub event loop and BackgroundService handling real-time events, presence detection, and cleanup jobs

3.2 Use Case Summary

Use Case	Actor(s)	Description
Register / Create Account	Guest, User	Create account with username, email, password; or via Google/GitHub OAuth
Login / Logout	User	Authenticate via JWT; on login ChatHub.OnConnectedAsync() fires UserConnected in PresenceService
Update Profile / Avatar	User	Change display name, bio, and avatar image stored in Azure Blob; UserProfile updated
Search Users	User	Full-text search on UserName and DisplayName via UserController.SearchUsers()
Send Direct Message	User	ChatHub.SendDirectMessage(receiverId, content) — persists Message entity, pushes via SignalR
Receive Real-Time Message	User, System	Hub calls Clients.User(receiverId).SendAsync('ReceiveMessage') instantly on message send
View Message History	User	GET /api/messages/direct/{userId} — paginated list of Message entities sorted by SentAt
Edit Message	User	PUT /api/messages/{id} — updates Content, sets IsEdited = true, EditedAt = DateTime.UtcNow
Delete Message	User	DELETE /api/messages/{id} — soft delete: IsDeleted = true, content replaced with placeholder
Search Messages	User	GET /api/messages/search?q=keyword — EF Core LIKE query on Message.Content for a conversation
Create Room / Group Chat	User	POST /api/rooms — creates ChatRoom entity; creator added as RoomMember with Role = ADMIN
Join / Leave Room	User	AddMember() / LeaveRoom() via Hub or REST; ChatHub.JoinRoom() calls Groups.AddToGroupAsync()

Use Case	Actor(s)	Description
Send Room Message	User	ChatHub.SendRoomMessage(roomId, content) — Hub broadcasts to Clients.Group(roomId.ToString())
View Room Members	User	GET /api/rooms/{roomId}/members — returns IList<RoomMember> with roles and join dates
Manage Room (Update/Delete)	User (Admin role)	PUT/DELETE /api/rooms/{id} — room admin updates name, description, or deletes the room
Browse Public Rooms	User, Guest	GET /api/rooms/public — returns IList<ChatRoom> where RoomType = PUBLIC and IsActive = true
See Online / Offline Status	User, System	PresenceService.IsUserOnline(userId) checked per user; online list pushed via SignalR on connect/disconnect
Typing Indicator	User, System	ChatHub.TypingIndicator(recipientId, isTyping) pushes real-time typing event to recipient's connection
Mark Message Read	User	ChatHub.MarkMessageRead(messageId) — sets IsRead = true, ReadAt = DateTime.UtcNow, pushes read receipt to sender
View Unread Message Count	User	GET /api/messages/unread-count/{userId} — COUNT query on Message where IsRead = false
Upload Image / File	User	POST /api/media/upload — stores IFormFile in Azure Blob; returns MediaFile entity with BlobUrl
View / Download Media File	User	GET /api/media/{fileId}/sas-url — returns time-limited SAS URL for secure Blob access
Send Media Message	User, System	ChatHub.SendMediaMessage() includes MediaUrl and MessageType (IMAGE/FILE/AUDIO) in persisted Message
View Room Media Gallery	User	GET /api/media/room/{roomId} — IList<MediaFile> filtered by RoomId ordered by UploadedAt
Receive Push Notification	User, System	Notification entity created; SignalR pushes badge count update to recipient's active connections
View Notification Centre	User	GET /api/notifications/{userId} — paginated IList<Notification> ordered by SentAt descending
Mark Notifications Read	User	PUT /api/notifications/{id}/read or PUT /api/notifications/read-all — sets IsRead = true
Receive Mention / Tag Alert	User, System	Message parser detects @username; Notification created with Type = MENTION and RelatedId = messageId
Manage Users (Admin)	Admin	GET/PUT/DELETE /api/users — suspend (IsActive=false) or permanently delete any user account
View All Rooms / Messages	Admin	Admin-scoped GET endpoints return all ChatRoom and Message entities platform-wide

Use Case	Actor(s)	Description
View Platform Analytics	Admin	Aggregate EF Core queries: total users, messages sent today, active rooms, connection count
Send Platform Notification	Admin	POST /api/notifications/broadcast — NotificationService.SendBulk() to all user IDs
View Audit Logs	Admin	GET /api/admin/audit-logs — timestamped AuditLog EF Core entity for all significant actions
Auto Online/Offline Detection	System	ChatHub.OnConnectedAsync() / OnDisconnectedAsync() fire UserConnected/UserDisconnected in PresenceService
Cleanup Expired Media	System	IHostedService daily job: DELETE MediaFile where ExpiresAt < DateTime.UtcNow + BlobClient.DeleteBlob()

4. Class Diagrams — All Services (.NET)

ConnectHub follows the EShoppingZone reference architecture adapted for C# and ASP.NET Core with SignalR. Each service has an EF Core Entity, a Repository Interface, a Service Interface + Implementation, and an API Controller (IActionResult). The ChatHub inherits from Hub<T> (Microsoft.AspNetCore.SignalR) and is the real-time layer sitting above the service layer. PresenceService is registered as AddSingleton (shared across all connections). All other services are AddScoped.

4.1 User / Auth-Service

The User/Auth-Service manages identity and authentication. JWT is configured via AddAuthentication().AddJwtBearer() in Program.cs. GitHub and Google OAuth are added via AddGitHub() / AddGoogle(). On successful login, the JWT sub claim carries UserId. ChatHub uses Context.UserIdentifier (mapped to UserId) for per-user targeted SignalR message delivery via Clients.User(userId). The SetOnlineStatus() method is called from ChatHub lifecycle events.

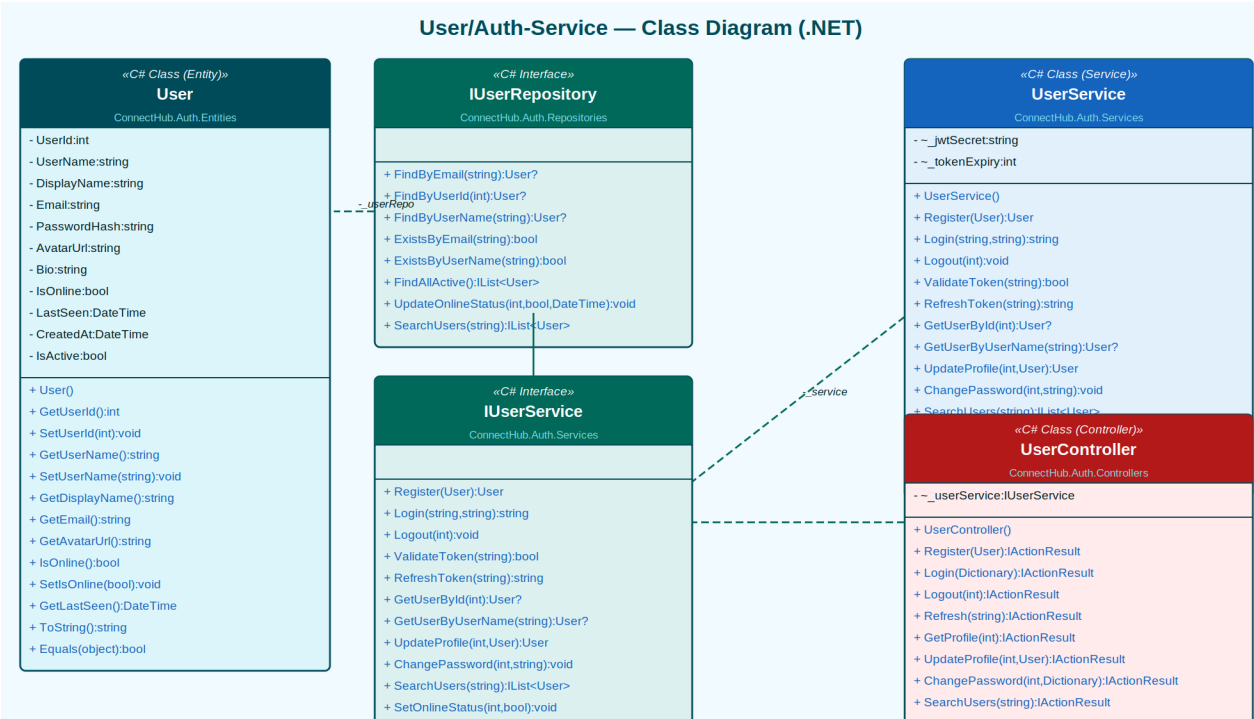


Figure 2: User/Auth-Service — Class Diagram (.NET / C#)

Component	Type	Responsibility
User	C# Class (EF Core Entity)	UserId, UserName, DisplayName, Email, PasswordHash, AvatarUrl, Bio, IsOnline (bool), LastSeen (DateTime), CreatedAt, IsActive
IUserRepository	C# Interface	FindByEmail(), FindByUserId(), FindByUserName(), ExistsByEmail/UserName(), FindAllActive(), UpdateOnlineStatus(), SearchUsers() — all Task<T>
IUserService	C# Interface	Register(), Login(), Logout(), ValidateToken(), RefreshToken(), GetUserById(), GetUserByUserName(), UpdateProfile(), ChangePassword(), SearchUsers(), SetOnlineStatus(), GetAllActiveUsers(), DeactivateAccount()
UserService	C# Class	Implements IUserService; uses PasswordHasher<User> for bcrypt hashing; AddAuthentication().AddJwtBearer() tokens issued on Login()

Component	Type	Responsibility
UserController	C# Class (ControllerBase)	[ApiController][Route("api/users")] — POST register/login/logout, PUT profile/password, GET search/active/byId, DELETE deactivate

4.2 Message-Service

The Message-Service handles all message persistence and retrieval. Message entities use nullable ReceiverId (for direct messages) and nullable RoomId (for room messages) — exactly one is populated per message. Soft delete (IsDeleted = true) preserves message history while hiding content. GetRecentChats() returns the most recent message per conversation partner, enabling the chat contact list sidebar. EF Core indexes on (SenderId, ReceiverId) and (RoomId, SentAt) optimise common query patterns.

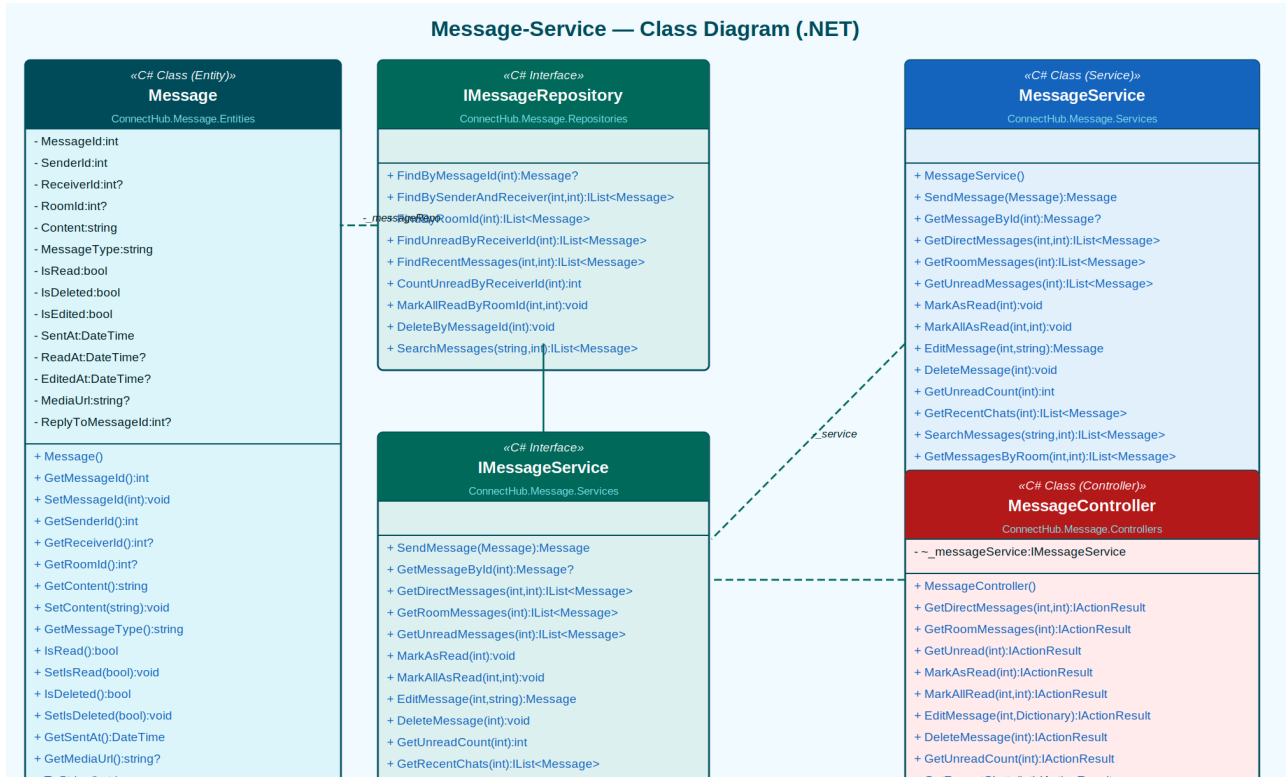


Figure 3: Message-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
Message	C# Class (EF Core Entity)	MessageId, SenderId, ReceiverId (int?), RoomId (int?), Content, MessageType (TEXT/IMAGE/FILE/AUDIO), IsRead (bool), IsDeleted (bool), IsEdited (bool), SentAt, ReadAt (DateTime?), EditedAt (DateTime?), MediaUrl (string?), ReplyToMessageId (int?)
IMessageRepository	C# Interface	FindByMessageId(), FindBySenderAndReceiver(), FindByRoomId(), FindUnreadByReceiverId(), FindRecentMessages(), CountUnreadByReceiverId(), MarkAllReadByRoomId(), DeleteByMessageId(), SearchMessages() — all Task<T>
IMessageService	C# Interface	SendMessage(), GetMessageById(), GetDirectMessages(), GetRoomMessages(), GetUnreadMessages(), MarkAsRead(), MarkAllAsRead(), EditMessage(), DeleteMessage(),

Component	Type	Key Attributes / Methods
		GetUnreadCount(), GetRecentChats(), SearchMessages(), GetMessagesByRoom()
MessageService	C# Class	Implements IMessageService; EF Core indexes on (SenderId,ReceiverId) and (RoomId,SentAt); soft delete pattern on DeleteMessage()
MessageController	C# Class (ControllerBase)	[ApiController][Route("api/messages")] — GET direct/room/unread/recent/search, PUT markRead/edit, DELETE; no POST (messages sent via SignalR Hub)

4.3 ChatRoom-Service

The ChatRoom-Service manages group chat rooms and their memberships. RoomType (PUBLIC/PRIVATE/DIRECT) determines visibility and join behaviour. A RoomMember dependent entity tracks each user's membership with a Role (ADMIN/MODERATOR/MEMBER) and JoinedAt timestamp. ChatHub.JoinRoom() calls both IChatRoomService.AddMember() and SignalR Groups.AddToGroupAsync(connectionId, roomId.ToString()) to enable group broadcast delivery. MaxMembers (default 500) is enforced in AddMember().

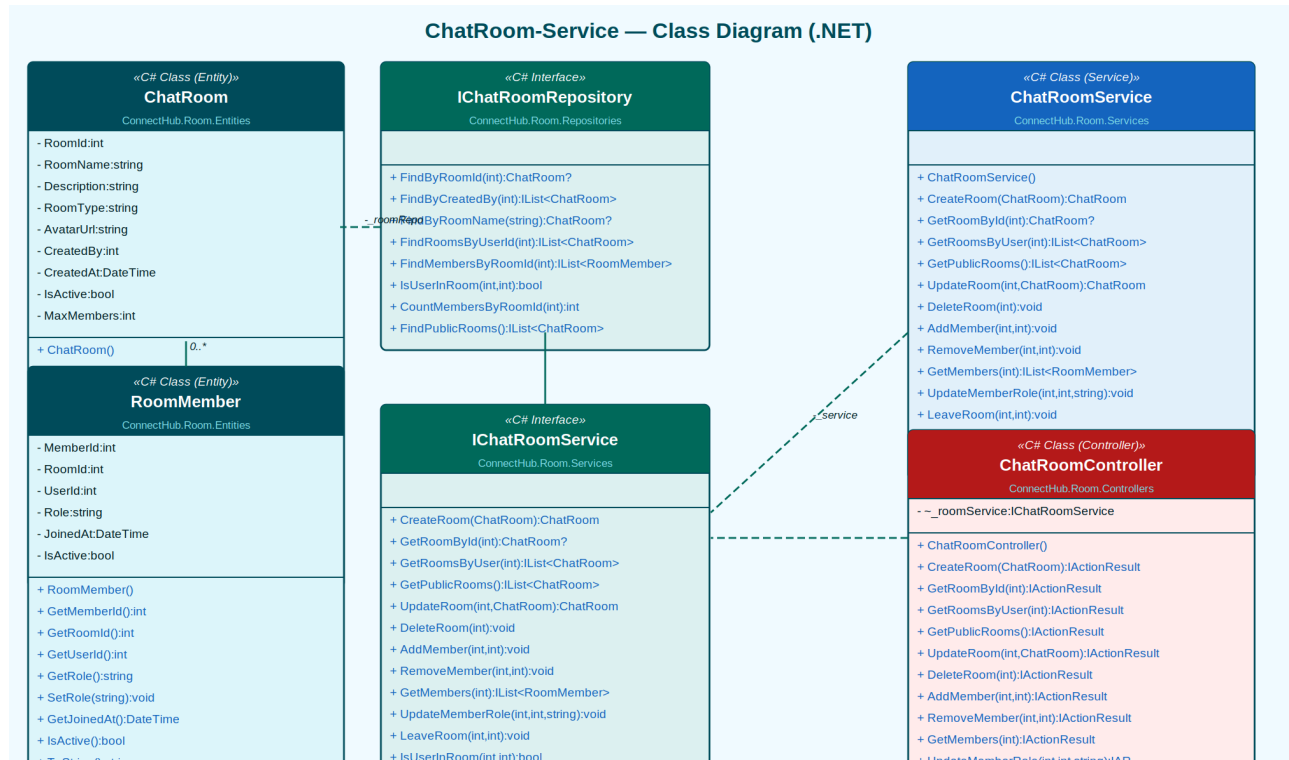


Figure 4: ChatRoom-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
ChatRoom	C# Class (EF Core Entity)	RoomId, RoomName, Description, RoomType (PUBLIC/PRIVATE/DIRECT), AvatarUrl, CreatedBy (FK to User), CreatedAt, IsActive, MaxMembers (int)
RoomMember	C# Class (EF Core Entity)	MemberId, RoomId (FK), UserId (FK), Role (ADMIN/MODERATOR/MEMBER), JoinedAt, IsActive — composite index on (RoomId, UserId)
IChatRoomRepository	C# Interface	FindByRoomId(), FindByCreatedBy(), FindByRoomName(), FindRoomsByUserId(), FindMembersByRoomId(), IsUserInRoom(), CountMembersByRoomId(), FindPublicRooms() — all Task<T>
IChatRoomService	C# Interface	CreateRoom(), GetRoomById(), GetRoomsByUser(), GetPublicRooms(), UpdateRoom(), DeleteRoom(), AddMember(), RemoveMember(), GetMembers(),

Component	Type	Key Attributes / Methods
		UpdateMemberRole(), LeaveRoom(), IsUserInRoom(), GetMemberCount()
ChatRoomController	C# Class (ControllerBase)	[ApiController][Route("api/rooms")] — POST create/addMember, GET byId/byUser/public/members, PUT update/memberRole, DELETE room/removeMember/leave

4.4 SignalR ChatHub & Presence-Service

ChatHub is the real-time communication core. It inherits from Hub (Microsoft.AspNetCore.SignalR NuGet) and is registered in Program.cs via builder.Services.AddSignalR() and app.MapHub<ChatHub>("/hubs/chat"). IPresenceService is registered as AddSingleton so the same ConcurrentDictionary<int, HashSet<string>> instance is shared across all Hub connections and API controllers. This enables any service to query online status without a database round-trip.

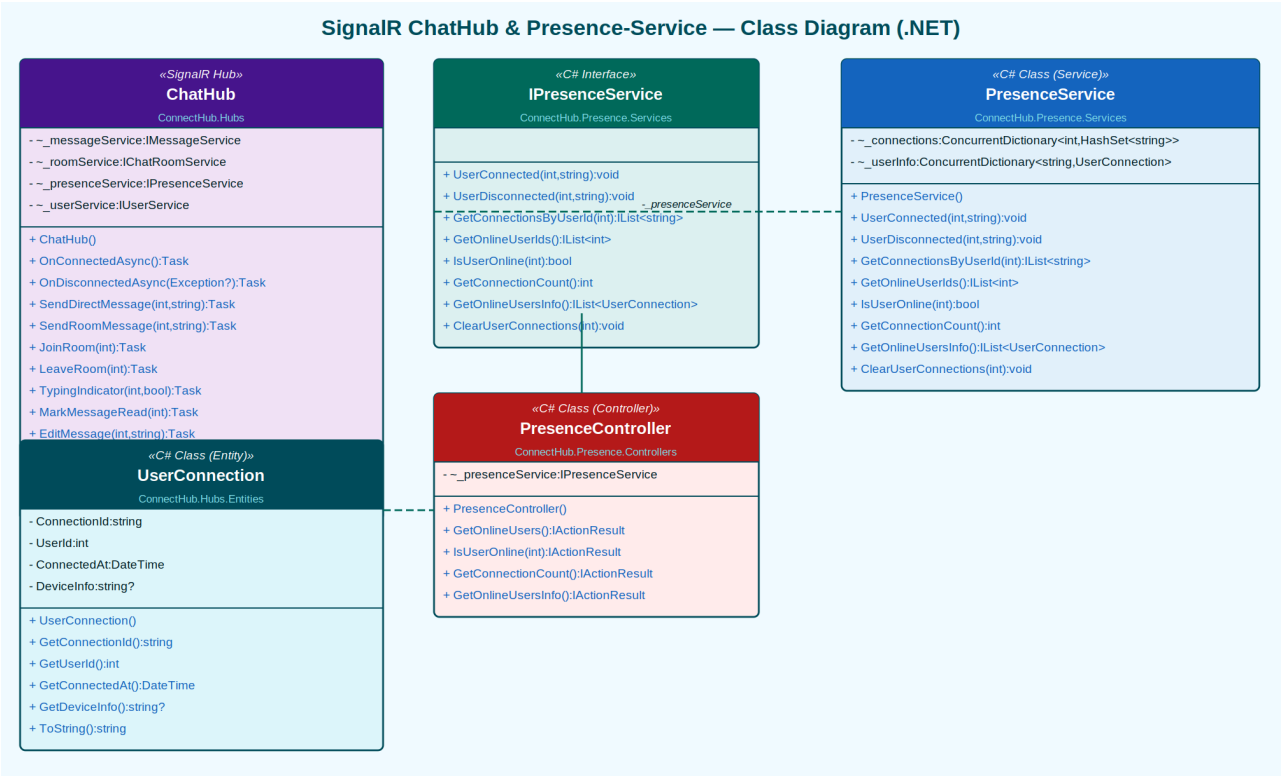


Figure 5: SignalR ChatHub & Presence-Service — Class Diagram (.NET / C#)

Component	Type	Key Methods / Notes
ChatHub	C# Class (Hub)	Inherits Hub; registered via <code>app.MapHub<ChatHub>("/hubs/chat")</code> ; all Hub methods return Task; Context.UserIdIdentifier maps to UserId via IUserIdProvider
OnConnectedAsync()	Hub Lifecycle	Calls <code>IPresenceService.UserConnected(userId, connectionId)</code> ; broadcasts online status to all clients; re-joins user to their rooms via <code>Groups.AddToGroupAsync()</code>
OnDisconnectedAsync()	Hub Lifecycle	Calls <code>IPresenceService.UserDisconnected(userId, connectionId)</code> ; broadcasts offline if no remaining connections for that user
SendDirectMessage(int, string)	Hub Method	Persists Message via <code>IMessageService.SendMessage()</code> ; calls

Component	Type	Key Methods / Notes
		Clients.User(receiverId).SendAsync('ReceiveMessage', message)
SendRoomMessage(int, string)	Hub Method	Persists Message with roomId; calls Clients.Group(roomId.ToString()).SendAsync('ReceiveRoomMessage', message)
TypingIndicator(int, bool)	Hub Method	Calls Clients.User(recipientId).SendAsync('UserTyping', senderId, isTyping) — no DB persistence
PresenceService	C# Class (Singleton)	ConcurrentDictionary<int, HashSet<string>> for connectionId tracking; ConcurrentDictionary<string, UserConnection> for connection metadata — thread-safe, in-memory
IPresenceService	C# Interface	UserConnected(), UserDisconnected(), GetConnectionsByUserId(), GetOnlineUserIds(), IsUserOnline(), GetConnectionCount(), GetOnlineUsersInfo(), ClearUserConnections()
PresenceController	C# Class (ControllerBase)	[ApiController][Route("api/presence")] — GET onlineUsers, isOnline(userId), connectionCount, onlineUsersInfo

4.5 Notification-Service

The Notification-Service dispatches in-app and email notifications. When a new Notification is persisted, the service immediately calls the SignalR Hub via IHubContext<ChatHub> to push the updated unread badge count to the recipient's active connections. MailKit (MimeKit/MailKit NuGet) sends email notifications to users who are offline. SendBulk() is used by Admin for broadcast platform notifications.

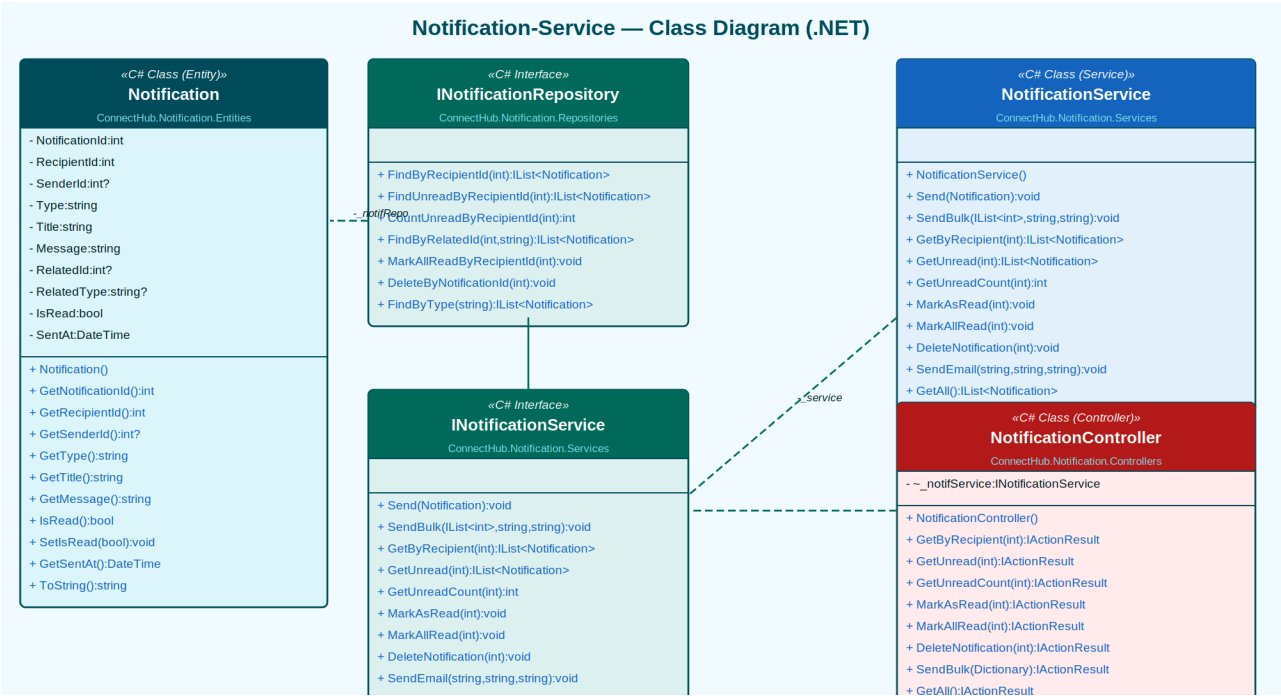


Figure 6: Notification-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
Notification	C# Class (EF Core Entity)	NotificationId, RecipientId, SenderId (int?), Type (MESSAGE/MENTION/ROOM_INVITE/ROLE_CHANGE/PLATFORM), Title, Message, RelatedId (int?), RelatedType (string?), IsRead (bool), SentAt
INotificationRepository	C# Interface	FindByRecipientId(), FindUnreadByRecipientId(), CountUnreadByRecipientId(), FindByRelatedId(), MarkAllReadByRecipientId(), DeleteByNotificationId(), FindByType() — all Task<T>
INotificationService	C# Interface	Send(), SendBulk(), GetByRecipient(), GetUnread(), GetUnreadCount(), MarkAsRead(), MarkAllRead(), DeleteNotification(), SendEmail(), GetAll()
NotificationService	C# Class	After Send(), calls IHubContext<ChatHub>.Clients.User(recipientId).Send Async('NotificationCount', unreadCount) for real-time badge update; email via MailKit for offline users

Component	Type	Key Attributes / Methods
NotificationController	C# Class (ControllerBase)	[ApiController][Route("api/notifications")] — GET byRecipient/unread/unreadCount/all, PUT markAsRead/markAllRead, DELETE, POST sendBulk

4.6 File / Media-Service

The Media-Service handles file uploads via IFormFile (ASP.NET Core multipart form). Files are uploaded to Azure Blob Storage using Azure.Storage.Blobs (BlobServiceClient.GetContainerClient().UploadBlobAsync()). The MediaFile EF Core entity stores metadata; the actual binary is in Blob. GenerateSasUrl() generates a time-limited SAS token (BlobSasBuilder with ExpiresOn = DateTime.UtcNow.AddHours(1)) for secure download without a public Blob container. An IHostedService cleans up expired files daily.

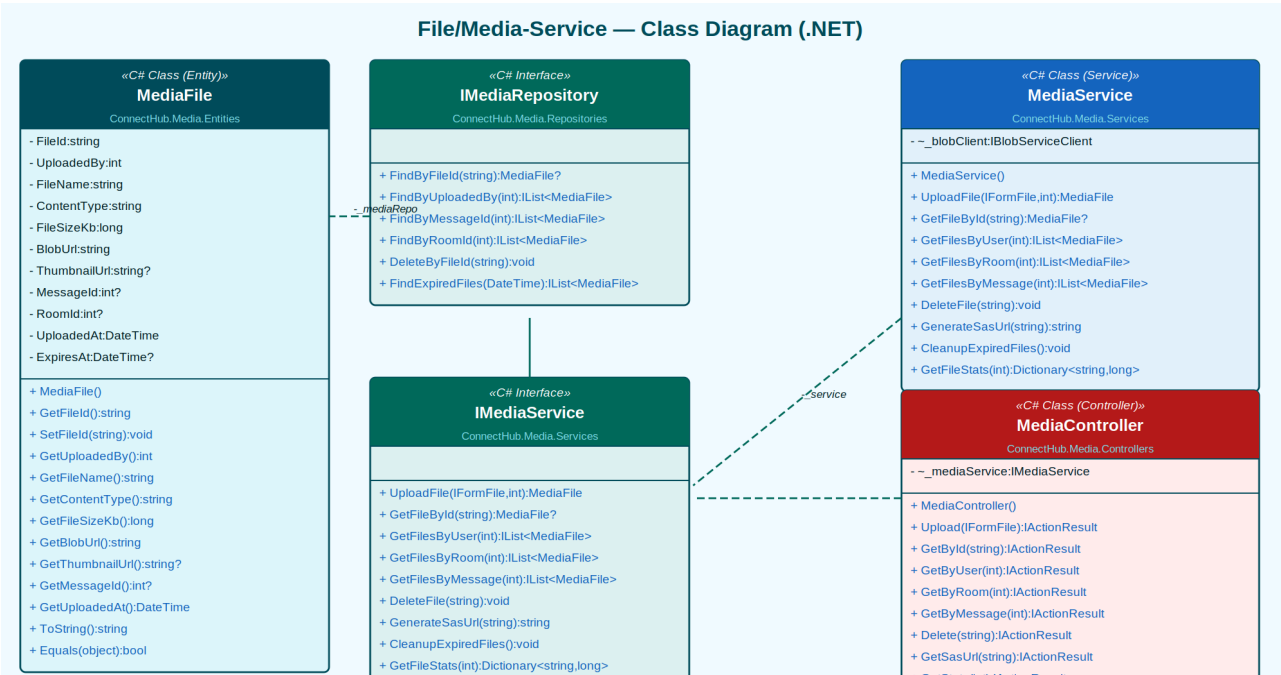


Figure 7: File/Media-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
MediaFile	C# Class (EF Core Entity)	Field (Guid as string), UploadedBy (FK to User), FileName, ContentType, FileSizeKb (long), BlobUrl, ThumbnailUrl (string?), MessageId (int?), RoomId (int?), UploadedAt, ExpiresAt (DateTime?)
IMediaRepository	C# Interface	FindByFileId(), FindByUploadedBy(), FindByMessageId(), FindByRoomId(), DeleteByFileId(), FindExpiredFiles(DateTime) — all Task<T>
IMediaService	C# Interface	UploadFile(IFormFile, int), GetFileById(), GetFilesByUser(), GetFilesByRoom(), GetFilesByMessage(), DeleteFile(), GenerateSasUrl() (BlobSasBuilder), CleanupExpiredFiles() (IHostedService), GetFileStats()
MediaService	C# Class	Uses Azure.Storage.Blobs BlobServiceClient injected via IOptions<AzureBlobOptions>; UploadFile() streams IFormFile directly to BlobClient.UploadAsync() without temp disk write

Component	Type	Key Attributes / Methods
MediaController	C# Class (ControllerBase)	[ApiController][Route("api/media")] — POST upload, GET byId/byUser/byRoom/byMessage/sasUrl, DELETE, GET stats

4.7 Website-Controller (MVC Layer)

The Website-Controller layer uses ASP.NET Core MVC with Razor Views (.cshtml). ChatController handles the full user-facing journey while AdminController provides the admin panel. Both inherit from Controller and inject services via constructor. The Razor views include the SignalR JavaScript client (microsoft/signalr npm package or CDN) which connects to /hubs/chat and listens for Hub events to update the UI in real time without page refresh.

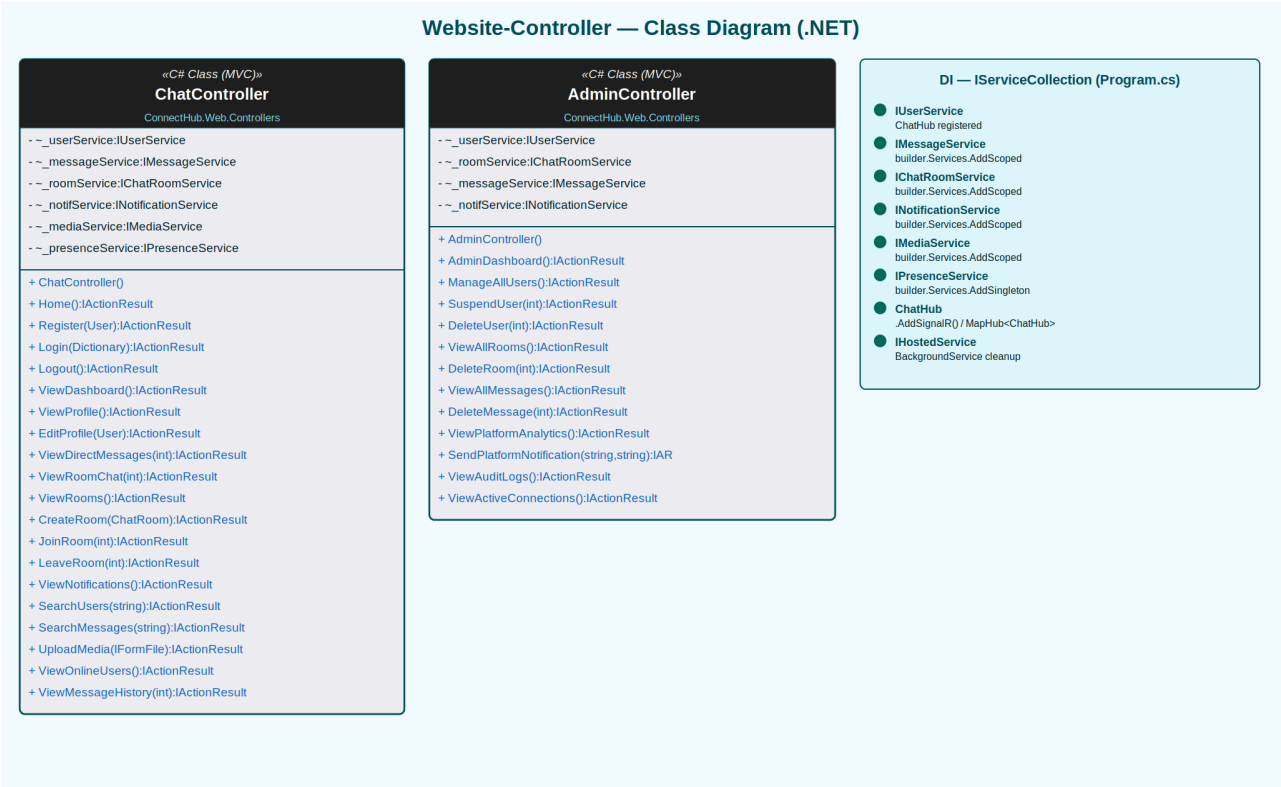


Figure 8: Website-Controller — Class Diagram (.NET / ASP.NET Core MVC)

Controller	Type	Key Methods
ChatController	C# Class (Controller)	Home(), Register(), Login(), Logout(), ViewDashboard(), ViewProfile(), EditProfile(), ViewDirectMessages(int), ViewRoomChat(int), ViewRooms(), CreateRoom(), JoinRoom(int), LeaveRoom(int), ViewNotifications(), SearchUsers(string), SearchMessages(string), UploadMedia(IFormFile), ViewOnlineUsers(), ViewMessageHistory(int)
AdminController	C# Class (Controller)	AdminDashboard(), ManageAllUsers(), SuspendUser(int), DeleteUser(int), ViewAllRooms(), DeleteRoom(int), ViewAllMessages(), DeleteMessage(int), ViewPlatformAnalytics(), SendPlatformNotification(string, string), ViewAuditLogs(), ViewActiveConnections()

5. Microservices Architecture Overview

ConnectHub is structured as independently deployable ASP.NET Core Web API projects plus one MVC web project. SignalR is hosted in the same Web project or a dedicated Hub API project. PresenceService is AddSingleton so its in-memory connection map is shared process-wide. For multi-server deployments, IPresenceService can be backed by Redis (StackExchange.Redis) via SignalR Redis Backplane (AddSignalR().AddStackExchangeRedis()).

Microservice (.NET Project)	Namespace	Primary Domain
ConnectHub.Auth.API	ConnectHub.Auth	User accounts, JWT Bearer, GitHub/Google OAuth2, online status
ConnectHub.Message.API	ConnectHub.Message	Message CRUD, direct/room messages, read receipts, search
ConnectHub.Room.API	ConnectHub.Room	ChatRoom + RoomMember management, join/leave, role management
ConnectHub.Hub.API	ConnectHub.Hubs	SignalR ChatHub, PresenceService (Singleton), real-time events
ConnectHub.Notification.API	ConnectHub.Notification	In-app notifications, MailKit email, unread badge via IHubContext
ConnectHub.Media.API	ConnectHub.Media	Azure Blob upload, SAS URL generation, IHostedService cleanup
ConnectHub.Web	ConnectHub.Web	ASP.NET Core MVC, Razor Views, SignalR JS client integration

6. Non-Functional Requirements

Category	Requirement
Real-Time Latency	SignalR WebSocket messages must reach all connected recipients within 200ms under normal load; Server-Sent Events fallback for clients that cannot upgrade to WebSockets
Scalability	SignalR Redis Backplane (AddSignalR().AddStackExchangeRedis()) enables horizontal scaling of the Hub API across multiple Kestrel instances behind YARP; PresenceService backed by Redis for multi-pod deployments
Security	JWT Bearer tokens on all API and SignalR Hub endpoints; [Authorize] on Hub methods via HubAuthorization middleware; input sanitised server-side before persistence; HTTPS enforced; CORS policy restricted to known frontend origins
Data Integrity	EF Core composite unique index on (SenderId, ReceiverId, SentAt) prevents duplicate message insertion; RoomMember unique index on (RoomId, UserId) prevents double membership
Availability	99.9% uptime SLA; SignalR automatic reconnect configured in the JS client (withAutomaticReconnect()); /health endpoints via Microsoft.Extensions.Diagnostics.HealthChecks
Message History	Messages are soft-deleted (IsDeleted = true) to preserve conversation history and audit trails; hard delete only by Admin after 30-day retention period
File Size Limits	Image uploads: max 10 MB; document uploads: max 50 MB; audio: max 25 MB — enforced at MediaController via RequestSizeLimit attribute
Audit Trail	All admin actions, account suspensions, room deletions, and broadcast notifications logged to AuditLog EF Core entity via ILogger<T> (Serilog)

7. Proposed Technology Stack (.NET)

Layer	Technology / NuGet Package
Backend Framework	ASP.NET Core 8 (Web API + MVC); .NET 8 SDK; Kestrel embedded server
Real-Time Messaging	Microsoft.AspNetCore.SignalR NuGet — WebSocket-based bidirectional communication; SignalR JS client (@microsoft/signalr npm)
SignalR Scale-Out	Microsoft.AspNetCore.SignalR.StackExchangeRedis NuGet — Redis backplane for multi-server SignalR deployment
Authentication	Microsoft.AspNetCore.Authentication.JwtBearer; AspNet.Security.OAuth.GitHub; Microsoft.AspNetCore.Authentication.Google; ASP.NET Core Identity (PasswordHasher<T>)
ORM / Data Access	Entity Framework Core 8 (Microsoft.EntityFrameworkCore); EF Core SQL Server (Microsoft.EntityFrameworkCore.SqlServer); EF Core Migrations
Database	SQL Server (primary per service via EF Core); Redis (StackExchange.Redis) for PresenceService connection map and SignalR backplane
File Storage	Azure.Storage.Blobs NuGet (Azure Blob Storage) for media files; BlobSasBuilder for SAS token generation; Azure CDN for static delivery
Email	MailKit NuGet (MimeKit/MailKit) for offline user email notifications via SMTP / AWS SES relay
Background Jobs	IHostedService / BackgroundService for daily expired media cleanup; Microsoft.Extensions.Hosting for scheduled task lifecycle
HTTP Clients	IHttpClientFactory typed clients with Microsoft.Extensions.Http.Polly NuGet for retry and circuit breaker on external API calls
Resilience	Polly (Microsoft.Extensions.Http.Polly NuGet) for retry, circuit breaker, and timeout policies on inter-service and external HTTP calls
Frontend	Razor Views (.cshtml) with Bootstrap 5 + @microsoft/signalr JS client; or React.js SPA with SignalR TypeScript client
API Gateway	YARP (Yarp.ReverseProxy NuGet) reverse proxy for routing, rate limiting, and JWT validation across all microservices
Logging	Microsoft.Extensions.Logging (ILogger<T>) with Serilog.AspNetCore NuGet sink to Seq or Azure Application Insights
API Documentation	Swashbuckle.AspNetCore NuGet for OpenAPI 3.0 auto-generated from controller XML comments and [ProducesResponseType] attributes
Containerisation	Docker multi-stage Dockerfile; Docker Compose for local development (with Redis + SQL Server); Kubernetes for production with HPA on Hub and Media services

Layer	Technology / NuGet Package
CI/CD	GitHub Actions: dotnet restore → dotnet build → dotnet test → docker build → push to ACR → kubectl rolling deploy

8. Glossary

Term	Definition
SignalR	ASP.NET Core library providing real-time, bidirectional WebSocket communication between server and clients; falls back to SSE or long-polling automatically
ChatHub	C# class inheriting Hub registered via <code>app.MapHub<ChatHub>('/hubs/chat')</code> ; server-side methods callable from SignalR JS client
IHubContext<T>	ASP.NET Core DI interface enabling server-side code (services, background jobs) to push messages to SignalR clients without being inside a Hub method
Hub Group	SignalR concept where all connections sharing a group name receive broadcast messages; <code>Groups.AddToGroupAsync(connectionId, roomId)</code> used for room chat
Clients.User(id)	SignalR method that targets all active WebSocket connections belonging to a specific user identifier — enables multi-device message delivery
PresenceService	AddSingleton C# class backed by <code>ConcurrentDictionary<int, HashSet<string>></code> tracking live connection IDs per UserId — thread-safe, in-memory
IHostedService	ASP.NET Core interface for background tasks; used for daily expired media file cleanup and scheduled notification dispatch
BlobSasBuilder	Azure.Storage.Blobs class generating time-limited Shared Access Signature URLs for secure file download without exposing the Blob storage account key
IFormFile	ASP.NET Core interface representing an uploaded multipart/form-data file; streamed directly to Azure Blob via <code>BlobClient.UploadAsync()</code>
SoftDelete	Pattern where <code>IsDeleted = true</code> hides message content without removing the row; preserves message ID references in replies and audit logs
MailKit	Cross-platform .NET email library (MimeKit/MailKit NuGet) providing SMTP support for offline user message notifications
IUserIdProvider	SignalR interface implemented to map the JWT sub claim (UserId) to <code>Context.UserId</code> for <code>Clients.User(userId)</code> targeted delivery
ConcurrentDictionary	Thread-safe .NET dictionary (<code>System.Collections.Concurrent</code>) used in <code>PresenceService</code> to track multiple simultaneous connections per user across threads
Redis Backplane	Microsoft.AspNetCore.SignalR.StackExchangeRedis NuGet enabling SignalR Hub message fan-out across multiple server instances via Redis pub/sub